

Examen-P2-SCD.pdf



DevGenius



Sistemas Concurrentes y Distribuidos



2º Grado en Ingeniería Informática



**Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada**



Escuela de
organización
Industrial

MÁSTER EN

Inteligencia Artificial & Data Management

MADRID

Formamos
talento para un futuro
Sostenible

saber más



Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

perdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

SISTEMAS CONCURRENTES Y DISTRIBUIDOS

Examen práctica 2 (17/11/2022)

Lea atentamente las siguientes NORMAS DE ESTE EXAMEN:

- En la identificación inicial se debe usar el código **ubu20exhome**.
- Si hay copia entre exámenes se anularán los dos exámenes implicados.
- Los ficheros deben subirse individualmente a la plataforma PRADO (**no** en un fichero comprimido).
- El nombre de dichos ficheros debe ser **exactamente** el que se indica en este folio de examen.
- Tiempo para este examen: 90 minutos.

1. Modificar el problema de los lectores-escritores de la siguiente manera (el archivo se debe llamar **"lec_esc_ex.cpp"**):

- En el procedimiento **"fin_escritura"**, la prioridad de acceso debe ser para los escritores. El número de procesos lectores será cinco y el de escritores, seis. Se debe crear una nueva hebra "limpiadora", que será un ciclo infinito y cuyo código será solamente un nuevo procedimiento del monitor que se llama "limpiar", seguido de un retardo aleatorio que simula la limpieza. El monitor tendrá una nueva variable permanente **"hacer_limpieza"** (booleana, inicializada a false).

- Hay que hacer una limpieza cada vez que haya un mínimo de 8 accesos completos de procesos lectores, en el momento que se haya completado el siguiente acceso de escritura (se considera un acceso completado cuando comienza la ejecución del correspondiente procedimiento **"fin_lectura"** o **"fin_escritura"**). Es decir, se deben contabilizar los accesos de los procesos lectores y cuando se hayan completado 8 de esos accesos (pudiendo haber accesos de escritura entre medias), se debe limpiar cuando se complete el siguiente acceso de escritura (independientemente de que pudiera haber más accesos completados de lectura antes del acceso del escritor).

- El proceso escritor que complete su acceso después de un mínimo de 8 accesos de lectura debe, en su procedimiento **"fin_escritura"**, despertar a la hebra "limpiadora" y quedarse bloqueado (antes de liberar a otro proceso lector o escritor) en espera de que la hebra limpiadora lo libere.

- En el procedimiento **"limpiar"**, se comprueba al principio el valor de la variable **"hacer_limpieza"**, si es false la hebra se bloquea, si es true (o si es desbloqueada la hebra), se debe indicar con un mensaje, cuántos accesos de lectura se han completado desde la última limpieza (valor mayor o igual a 8) e imprimir otro mensaje con el identificador del proceso escritor que la ha liberado. Después, se debe inicializar la cuenta de accesos de lectura y desbloquear al escritor que la liberó, que ya podrá completar su procedimiento **"fin_escritura"**.

2. Un programa concurrente está formado por 12 hebras que representan a clientes de un banco. El banco tiene 3 ventanillas de atención a clientes (0, 1 y 2) que solo atienden a un cliente en cada momento. Los clientes son fieles a la ventanilla que escogieron (aleatoriamente) la primera vez que fueron al banco. Por tanto, siempre van a la misma ventanilla. El tiempo de gestión en la ventanilla y la parte de resto de código se simulan con retardos aleatorios. Implementar dicho programa (el archivo se deberá llamar **"ejercicio2.cpp"**) con un monitor de acuerdo al siguiente esquema. **Incluir mensajes que permitan seguir la traza del programa.**

Monitor Banco	Procedure acceso (vent: integer)	Procedure fin_acceso (vent: integer)
var	begin	begin
variables permanentes	si no está libre vent → esperar (bloqueo)	actualizar estado (ventanilla libre)
variables condición	actualizar estado (ventanilla ocupada)	liberar un proceso
	end	end

```
TipoHebra
var ventanilla: integer
begin
  decidir aleatoriamente el valor de ventanilla (entero entre {0, 1, 2})
  while (true) do
    begin
      Banco.acceso (ventanilla);
      {retardo aleatorio} (gestión en el banco)
      Banco.fin_acceso (ventanilla);
      {retardo aleatorio} (resto de código)
    end
  end
end
```

WUOLAH